LegalLens

Solution Proposal: a private, on-device legal document triage agent

ZeroDay OSS Hacktoberfest 2026, IIIT Dharwad · PS2: Agentic AI · Team TH3_B0Tz · October 2026

LegalLens is for preliminary document understanding and issue spotting only. It is not a lawyer, gives no legal advice, and never decides whether a clause is valid, legal or enforceable.

1 Summary

LegalLens reads a rental or employment agreement **entirely on the user's machine**. It explains clauses in plain language and flags obligations, rights, deadlines, penalties, termination terms and ambiguities. Every claim carries a verbatim quote from the document, checked by code. When the document is silent, or the question needs a lawyer, LegalLens says so instead of guessing.

Design rule: **the model proposes, code disposes.** A local Gemma model drafts findings as JSON. Deterministic code verifies every quote, number and date, refuses verdict questions before the model is called, and keeps all data on the device.

Item	Choice
Model	Gemma 4 E4B (edge-class), Q4_K_M 4-bit GGUF, 4.98 GB, Apache-2.0; fallback E2B
Runtime	llama.cpp llama-server on 127.0.0.1 (loopback only), 8k context
Retrieval	BM25 + local MiniLM-class embeddings, FAISS index on disk
UI	Gradio on 127.0.0.1, analytics off, no CDN
Stack	Python 3.11+, pinned dependencies, MIT licence
Scope	English; text PDF and .txt; rental and employment agreements

2 Problem and Users

People sign rental agreements and job offers without fully understanding them, and a lawyer for every first question is not practical. Generic chatbots invent clauses, answer from general knowledge and sound certain. Uploading a private contract to a cloud service is a privacy risk in itself.

- **Users:** tenants, new employees and small-business owners reviewing an agreement before signing.
- **Need:** what the document actually says, what it costs them, what is missing or vague, and what to ask a professional.
- **Non-goals:** judging validity or enforceability, naming laws as conclusions, recommending “sign” or “don’t sign”, and any cloud call during processing.

3 Proposed Solution

The user opens a document in the local UI and gets:

- **Grounded summary:** every sentence cites the passages it rests on.
- **Prioritised findings:** Critical, Important and Informational tiers, ranked by a severity score computed in code. Each finding shows its source passage beside the explanation.

- **Labels:** STATED (the text says it), INTERPRETATION (our reading) and CONCERN (worth asking about).
- **Coverage report:** which clause types were not found, for example “no termination clause”.
- **Document Q&A:** cited answers, or “the document does not appear to say anything about this” when it is silent.
- **Escalation panel:** verdict questions are declined with specific advice on which clause to bring, what to ask and which facts matter.

4 Architecture

Six stages, all on the user's device. There is no network access at run time; downloads happen only at setup.

#	Stage	What happens
1	Ingest	pypdf / pdfplumber produce text and a page map, then clause-aware chunks with stable IDs (p3.c2) and offsets. Empty, encrypted or scanned files get a clear warning
2	Index	BM25 and local embeddings feed a FAISS index in one session directory
3	Analyse	Gemma extracts findings as JSON. Document text is passed as delimited data, never as instructions. Malformed JSON is retried twice, then skipped
4	Verify	Code only: the quote must be a substring of the cited passage after normalisation, and numbers and dates in the explanation must appear in the quote. Failures are dropped or marked unverified
5	Safety	Labels, ambiguity and jurisdiction flags, silence logic, verdict refusal, a forbidden-phrase filter and a fixed disclaimer
6	UI	127.0.0.1 only: summary, findings, clause view with the source beside each explanation, and Q&A

5 Grounding and Safety

- **Verified quotes:** an exact normalised match passes. A near match (≥ 0.95) is replaced with the real source span, and anything else is dropped. The summary is checked sentence by sentence the same way.
- **No number drift:** “30 days” explained as “3 months” is caught and hidden.
- **Honest silence:** a low retrieval score with no verified citation gives a fixed silent reply, never an answer from general knowledge.
- **Refusal in code:** “is this enforceable?” and “should I sign?” are classified and declined before Gemma is called.
- **Uncertainty:** vague terms (“reasonable”, “sole discretion”), blanks and conflicting clauses are flagged as ambiguous. Deposits, notice periods and non-competes are flagged as jurisdiction-dependent, with no conclusion.
- **Prompt injection:** document text is untrusted data in delimited blocks, and the output is a fixed JSON schema. Instruction-like text is ignored and flagged to the user.

6 Privacy and Proof

Nothing private leaves the device: not the file, text, chunks, embeddings, summaries, answers or questions. We prove this in three independent ways:

Layer	Test
1. Cannot connect	Full analysis with the network adapter off (Windows), or under docker --network none / unshare -rn
2. Does not try	A connection monitor around a normal run records zero non-loopback connections
3. No code that connects	A dependency audit for HTTP clients, telemetry and update checks, with every hit reviewed in THREAT_MODEL.md

In addition, telemetry kill-switches are set in code (HF_HUB_OFFLINE, TRANSFORMERS_OFFLINE, GRADIO_ANALYTICS_ENABLED=False, DO_NOT_TRACK). Logs hold timings and counts only, never document text. All session files live in one directory with a “clear session” control.

7 Gemma Integration

Gemma does the central task: clause extraction, plain-language explanation, labelling, summary and Q&A, all locally. It is called through the chat endpoint so the Gemma chat template is applied. E4B at 4-bit fits a 16 GB laptop alongside the UI and the embedder. Retrieval limits Q&A prompts to the top 3 passages, which keeps answers under about 30 s on CPU. The fallback ladder is Q3_K_S, then E2B, each re-measured. RAM, load time and tokens per second are reported with the exact hardware.

8 Mandatory Requirements

ID	Requirement	Delivered by	Status
L1	Local ingestion (PDF / .txt)	ingest.py	Pending (B)
L2	On-device extraction and interpretation	ingest.py + analyze.py	Analysis landed
L3	Plain-language summaries and explanations	analyze.py prompts	Landed
L4	Obligations, rights, deadlines, penalties, termination	Finding kinds + coverage	Landed
L5	Answers backed by identifiable passages	Passage IDs + verify.py	Partial (fallback in analyze.py)
L6	Stated vs interpretation	Labels + downgrade rules	Partial
L7	Ambiguity, missing evidence, jurisdiction	Flags + silence logic	Silence landed
L8	Refuse verdicts, recommend a lawyer	Classifier + escalation	Refusal landed
L9	Demonstrable privacy	privacy/ three-layer proof	Pending (B)
L10	Meaningful Gemma use, reproducible OSS	llm.py, docs, pinned deps	Landed; LICENSE pending

Gates G1–G5 (public repo and licence, no paid services or secrets, a real system, datasets disclosed) and L1–L4 (local Gemma, nothing leaves the device, no fabrication, not legal advice) are designed in from the start, because a failed gate means the team is not ranked.

9 Evaluation Plan

- **Samples we write:** clear and vague rentals, standard and conflicting employment contracts, an injected document, a complex layout and malformed files. One held-out document and 1–2 public templates are never used for tuning.
- **Metrics:** quote-verification pass rate, citation accuracy, human-scored meaning accuracy (X/10 clauses), silence and refusal correctness, injection results, latency and peak RAM.

- **Today:** 20/20 deterministic tests pass, covering JSON handling, retries, analysis shapes, injection containment, refusal, the silent path and type contracts. An offline smoke demo runs without the model.

10 Team and Timeline

Lane	Owns
A: Gemma + analysis	llm.py, prompts, analyze.py: extraction, explanation, summary, Q&A, long documents
B: Ingest + privacy	ingest.py, retrieve.py, privacy/: parsing, chunking, index, offline run, monitor, audit, threat model
C: Grounding + safety	verify.py, safety.py, injection.py: verifier, labels, flags, refusal, escalation, filter
D: UI, eval, docs	app/, bench/, samples, README, CI, demo recording, submission

12-hour checkpoints: H1 model locked · H4 local Gemma, ingest and verifier working · H6 end-to-end on two document types, scope cuts decided · H9 safety cases pass and the offline run is recorded · H10.5 code freeze. Scope cuts, in order: service agreements, structured extraction, optional extras, CUAD, free-form Q&A.

11 Key Risks

Risk	Mitigation
Model invents quotes (score capped at 40)	The code verifier drops anything not in the source, summary included
Output reads as legal advice (cap)	Refusal in code, labels, a forbidden-phrase filter and a fixed disclaimer
A dependency phones home (cap)	Kill-switches, dependency audit, offline run and monitor log
Gemma too slow on laptops	Top-3 retrieval, E2B or lower-quant fallback, a recorded backup demo
Poor PDF extraction or long documents	Two parsers, clause-aware chunking, a coverage report and honest warnings

12 Limitations

- English only; text PDF and .txt. Scanned PDFs are detected and reported, not OCR'd.
- Tested on rental and employment agreements only.
- A relevant clause that retrieval ranks low yields a silent answer, never a wrong one. This is deliberate.
- Privacy covers the software, not a compromised OS or a malicious model file.

13 Main Technical Contribution

A reusable grounding layer for on-device legal assistants. It combines clause-aware passage IDs, a deterministic quote-and-number verifier applied to findings and summaries alike, refusal enforced in code rather than in the prompt, and a reproducible three-layer privacy proof. Together these show that a small edge model can be used on legal text, because code, not the model, decides what reaches the user.